

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management

Systems COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate

7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze
8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear processes
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Question 1: SQL Queries Using Joins and Subqueries

Explanation:

SQL queries are used to retrieve information from one or more database tables. When data is distributed across multiple tables, **JOIN** operations are used to combine related records based on a common attribute, such as a primary key and a foreign key. This helps retrieve meaningful information without storing duplicate data.

A **subquery** is a query written inside another SQL query. The inner query executes first, and its result is passed to the outer query. Subqueries are useful for comparing values, filtering records, and solving complex retrieval problems.

```
mysql> SELECT StudentName, DeptName
-> FROM Student
-> JOIN Department
-> ON Student.DeptID=Department.DeptID;
+-----+-----+
| StudentName | DeptName |
+-----+-----+
| Rahul       | CSE      |
| Kiran       | CSE      |
| Priya       | IT       |
| Sneha       | ECE      |
+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT StudentName
-> FROM Student
-> WHERE Marks>(
-> SELECT AVG(Marks)
-> FROM Student);
+-----+
| StudentName |
+-----+
| Priya       |
| Sneha       |
+-----+
2 rows in set (0.00 sec)
```

Logic Used:

- Use **JOIN** when data is required from multiple related tables.
- Use **Subqueries** when one query depends on the result of another query.
- Joins generally provide better performance and are easier to understand for relational databases.

Question 2: Relational Algebra Expressions

Explanation:

Relational algebra is the theoretical foundation of relational databases. It consists of mathematical operations used to manipulate relations (tables) and retrieve desired information.

The major relational algebra operators include:

- **Selection (σ):** Selects rows that satisfy a given condition.
- **Projection (π):** Selects specific columns from a table.
- **Join ($\bowtie$):** Combines related tuples from two relations.
- **Union ($\cup$):** Combines tuples from two compatible relations.
- **Difference ($-$):** Returns tuples present in one relation but not in another.
- **Intersection ($\cap$):** Returns tuples common to both relations.

```
mysql> SELECT StudentName
-> FROM Student
-> WHERE DeptID=1;
+-----+
| StudentName |
+-----+
| Rahul       |
| Kiran       |
+-----+
2 rows in set (0.00 sec)
```

Logic Used:

Relational algebra provides a step-by-step procedure for retrieving data. SQL statements are based on these operations and produce equivalent results in practical database systems.

Question 3: Relational Schema Design and SQL Queries

Explanation:

A relational schema defines the structure of a database by specifying tables, attributes, primary keys, and foreign keys. Proper schema design ensures that data is organized efficiently and redundancy is minimized.

A **Primary Key** uniquely identifies each record in a table, while a **Foreign Key** establishes relationships between tables.

After designing the schema, SQL statements such as **SELECT**, **INSERT**, **UPDATE**, and **DELETE** are used to manage the stored data.

```
mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| StudentID | StudentName | DeptID | Marks |
+-----+-----+-----+-----+
|      101 | Rahul      |      1 |    85 |
|      102 | Priya      |      2 |    90 |
|      103 | Kiran      |      1 |    75 |
|      104 | Sneha      |      3 |    95 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> UPDATE Student
-> SET Marks=95
-> WHERE StudentID=103;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Logic Used:

- Create normalized tables.
- Define relationships using foreign keys.
- Retrieve and update data using SQL commands.

Question 4: Analysis of Nested Subqueries

Explanation:

A nested subquery contains one SQL query inside another. The database management system executes the innermost query first and uses its result in the outer query.

Execution Steps:

1. Execute the inner query.
2. Store the result temporarily.
3. Execute the outer query using the temporary result.
4. Display the final output.

```
mysql> SELECT StudentName
-> FROM Student
-> WHERE DeptID=(
-> SELECT DeptID
-> FROM Department
-> WHERE DeptName='CSE');
+-----+
| StudentName |
+-----+
| Rahul      |
| Kiran      |
+-----+
2 rows in set (0.00 sec)
```

Logic Used:

Nested subqueries simplify complex conditions where the output of one query is required as input to another query.

Question 5: Comparison of Joins and Subqueries

Explanation:

Both joins and subqueries are used to retrieve data from relational databases, but they differ in execution and performance.

Joins

- Retrieve data from multiple related tables.
- Execute faster for large datasets.
- Improve readability and efficiency.

```
mysql> SELECT StudentName,DeptName
-> FROM Student
-> JOIN Department
-> ON Student.DeptID=Department.DeptID;
+-----+-----+
| StudentName | DeptName |
+-----+-----+
| Rahul      | CSE      |
| Kiran      | CSE      |
| Priya      | IT       |
| Sneha      | ECE      |
+-----+-----+
4 rows in set (0.00 sec)
```

Subqueries

- Execute one query inside another.
- Useful for filtering and conditional comparisons.
- May require additional processing for large databases.

```
mysql> SELECT StudentName,
-> (
-> SELECT DeptName
-> FROM Department
-> WHERE Department.DeptID=Student.DeptID
-> )
-> AS Department
-> FROM Student;
+-----+-----+
| StudentName | Department |
+-----+-----+
| Rahul      | CSE      |
| Priya      | IT       |
| Kiran      | CSE      |
| Sneha      | ECE      |
+-----+-----+
4 rows in set (0.00 sec)
```

Evaluation:

For most relational database applications, joins are preferred because they optimize query execution and improve performance.

Question 6: Views for Security and Abstraction**Explanation:**

A **View** is a virtual table created using a SQL SELECT statement. It does not store data physically but displays data retrieved from one or more tables.

Views are used to:

- Restrict access to confidential information.
- Simplify complex SQL queries.
- Improve data security.
- Provide logical data abstraction.

```
mysql> CREATE VIEW Student_View AS
-> SELECT StudentID,StudentName
-> FROM Student;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> SELECT * FROM Student_View;
+-----+-----+
| StudentID | StudentName |
+-----+-----+
|      101 | Rahul       |
|      102 | Priya       |
|      103 | Kiran       |
|      104 | Sneha       |
+-----+-----+
4 rows in set (0.00 sec)
```

Logic Used:

Views allow users to access only the required information without exposing the complete database structure.

Question 7: Relational Algebra to SQL Conversion**Explanation:**

Relational algebra provides mathematical expressions for database operations, whereas SQL provides practical commands for implementation.

For example, a **Selection** operation in relational algebra is represented using the **WHERE** clause in SQL. Similarly, **Projection** corresponds to selecting required columns using the **SELECT** statement.

```
mysql> SELECT StudentName
-> FROM Student
-> WHERE Marks>80;
+-----+
| StudentName |
+-----+
| Rahul       |
| Priya       |
| Kiran       |
| Sneha       |
+-----+
4 rows in set (0.00 sec)
```

Logic Used:

Although the syntax differs, relational algebra and SQL represent the same database operations and produce equivalent outputs.

Question 8: SQL Query Optimization

Explanation:

Query optimization improves the efficiency of SQL statements by reducing execution time and minimizing resource consumption.

Common optimization techniques include:

- Replacing unnecessary subqueries with joins.
- Creating indexes on frequently searched columns.
- Using views for repeated complex queries.
- Selecting only the required columns instead of using **SELECT ***.
- Avoiding redundant calculations.

```
mysql> SELECT StudentName
-> FROM Student
-> WHERE DeptID IN
-> (
-> SELECT DeptID
-> FROM Department
-> WHERE DeptName='CSE'
-> );
+-----+
| StudentName |
+-----+
| Rahul       |
| Kiran       |
+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT StudentName
-> FROM Student
-> JOIN Department
-> ON Student.DeptID=Department.DeptID
-> WHERE DeptName='CSE';
+-----+
| StudentName |
+-----+
| Rahul       |
| Kiran       |
+-----+
2 rows in set (0.00 sec)
```

Evaluation:

Optimized queries execute faster, consume fewer system resources, and improve the overall performance of the database.

Question 9: Grouping, Aggregation, and Nested Queries

Explanation:

Grouping and aggregation are used to summarize large amounts of data.

Common aggregate functions include:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

The **GROUP BY** clause groups records based on common values before applying aggregate functions.

Nested queries can also be combined with aggregate functions to solve complex real-world database problems, such as identifying departments with above-average student marks.

```
mysql> SELECT DeptName,
-> COUNT(StudentID) AS TotalStudents,
-> AVG(Marks) AS AverageMarks
-> FROM Student
-> JOIN Department
-> ON Student.DeptID=Department.DeptID
-> GROUP BY DeptName;
+-----+-----+-----+
| DeptName | TotalStudents | AverageMarks |
+-----+-----+-----+
| CSE      | 2             | 90.0000     |
| IT       | 1             | 90.0000     |
| ECE      | 1             | 95.0000     |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Logic Used:

Grouping organizes similar records, while aggregate functions calculate statistical summaries for each group.

Question 10: Choosing Between Views, Joins, and Subqueries

Explanation:

Different SQL techniques are suitable for different database requirements.

- **Joins** are used when data must be retrieved from multiple related tables.
- **Subqueries** are used when one query depends on the result of another query.
- **Views** are used to simplify complex queries and provide secure access to selected data.

```
mysql> SELECT StudentName,DeptName
-> FROM Student
-> JOIN Department
-> ON Student.DeptID=Department.DeptID;
+-----+-----+
| StudentName | DeptName |
+-----+-----+
| Rahul       | CSE      |
| Kiran       | CSE      |
| Priya       | IT       |
| Sneha       | ECE      |
+-----+-----+
4 rows in set (0.00 sec)
```

Justification:

- Choose **Joins** for faster retrieval from multiple tables.
- Choose **Subqueries** for conditional or dependent queries.
- Choose **Views** for security, abstraction, and code reusability.

Selecting the appropriate approach improves database performance, enhances security, and simplifies query management.